Dossier technique de modélisation de Shazam

PERRET Loris
BONNEAU Baptiste

CLERMONT AUVERGNE INP Isima

Sommaire

Législation	3
Outils de modélisation	4
Diagramme de cas d'utilisation	5
Diagramme de classes et Architecture	6
Diagramme de base de données (MLD)	10
Diagramme d'état	11
Diagramme d'activité	18
Direction artistique	19

Législation

A-t-on le droit de programmer l'algorithme Shazam, de vendre notre réalisation et d'utiliser des œuvres musicales ?

À but éducatif ou personnel :

Refaire un algorithme tel que celui de Shazam nécessite de prendre des précautions, il nous est tout à fait possible de créer un algorithme de reconnaissance musicale similaire ou égal à Shazam si c'est dans un cadre personnel ou éducatif. Dans ce contexte, nous pouvons totalement utiliser des outils mathématiques comme la transformation de Fourier ou la création d'empreintes audio étant donné que ces concepts sont largement connus et ne peuvent donc pas être protégés.
Par contre, si l'on souhaite partager notre code ou rendre le projet accessible en ligne, cela pourrait poser des problèmes juridiques, même sans intention commerciale.

À but de diffusion :

Cependant, il faut prendre en compte les différents brevets déposés par Shazam si l'on souhaite utiliser notre algorithme à des fins commerciales. Ces brevets protègent les aspects spécifiques de l'algorithme. Reproduire leur algorithme sans leur autorisation au préalable pour entraîner des problèmes juridiques. Afin de ne pas avoir de problèmes, il faut s'assurer que notre algorithme reste original, unique et qu'il respecte ces brevets.
De plus, il faut s'assurer que notre application finale est aussi unique et qu'elle n'est une totale copie de Shazam dans le but d'éviter toute concurrence déloyale.

Données et œuvres musicales :

Dès lors que notre application est utilisée, que ce soit par nous lors du développement, lors des phases de tests de l'application ou par nos utilisateurs une fois l'application publiée, nous ne pouvons pas utiliser d'extraits musicaux sans avoir au préalable les autorisations nécessaires pour éviter d'enfreindre les droits d'auteur associés aux œuvres.

Outils de modélisation

Nous avons choisi d'intégrer dans notre modélisation plusieurs types diagrammes ainsi que des maquettes graphiques.

Diagramme de cas d'utilisation :

- Permet de représenter les fonctionnalités présentes sur l'application et de classer leur accessibilité en fonction des différents acteurs

Diagramme d'activité :

- Met en avant les flux d'activités et de décisions pour une fonctionnalité donnée

Diagramme d'état :

- Permet de représenter les différents états de l'application, notamment les états d'erreur. Permet aussi de connaître le déroulement des états de l'application lors de l'utilisation.

Diagramme MLD de base de données :

- Permet de représenter comment sont stockées les informations dans la base de données et de visualiser les différentes relations présentes entre ces entités

Diagramme de classes :

- Permet de connaître l'architecture et l'implémentation structurelle du code de l'application.

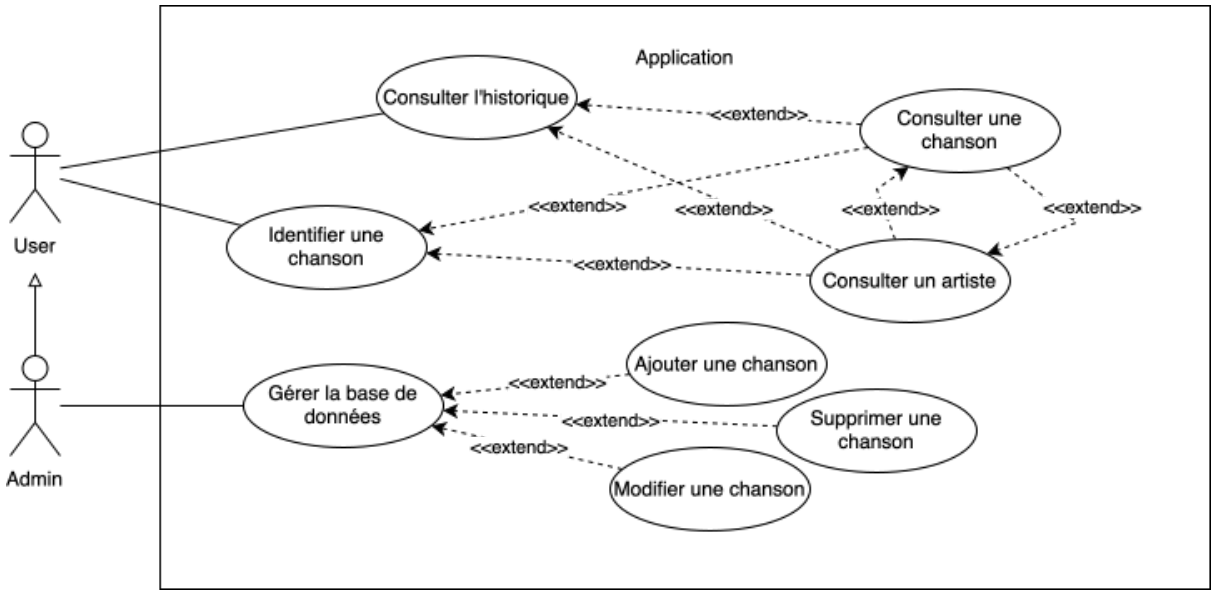
Diagramme de séquence :

- Illustre les échanges entre les différents systèmes ou objets dans un ordre temporel précis.

Pour réaliser ces diagrammes, nous avons utilisé plusieurs outils tels que Google Draw.io, StarUML, PlantUML. Nous avons fait le choix d'utiliser plusieurs outils pour nous adapter aux différents types de diagramme car chacun d'eux apporte des avantages et des inconvénients.

Enfin, pour la réalisation des maquettes graphiques, nous avons utilisé Figma.

Diagramme de cas d'utilisation



Le diagramme de cas d'utilisation représente les possibles actions qu'un utilisateur peut faire. On distingue deux types d'utilisateurs, les utilisateurs normaux et ceux ayant des droits administrateurs.

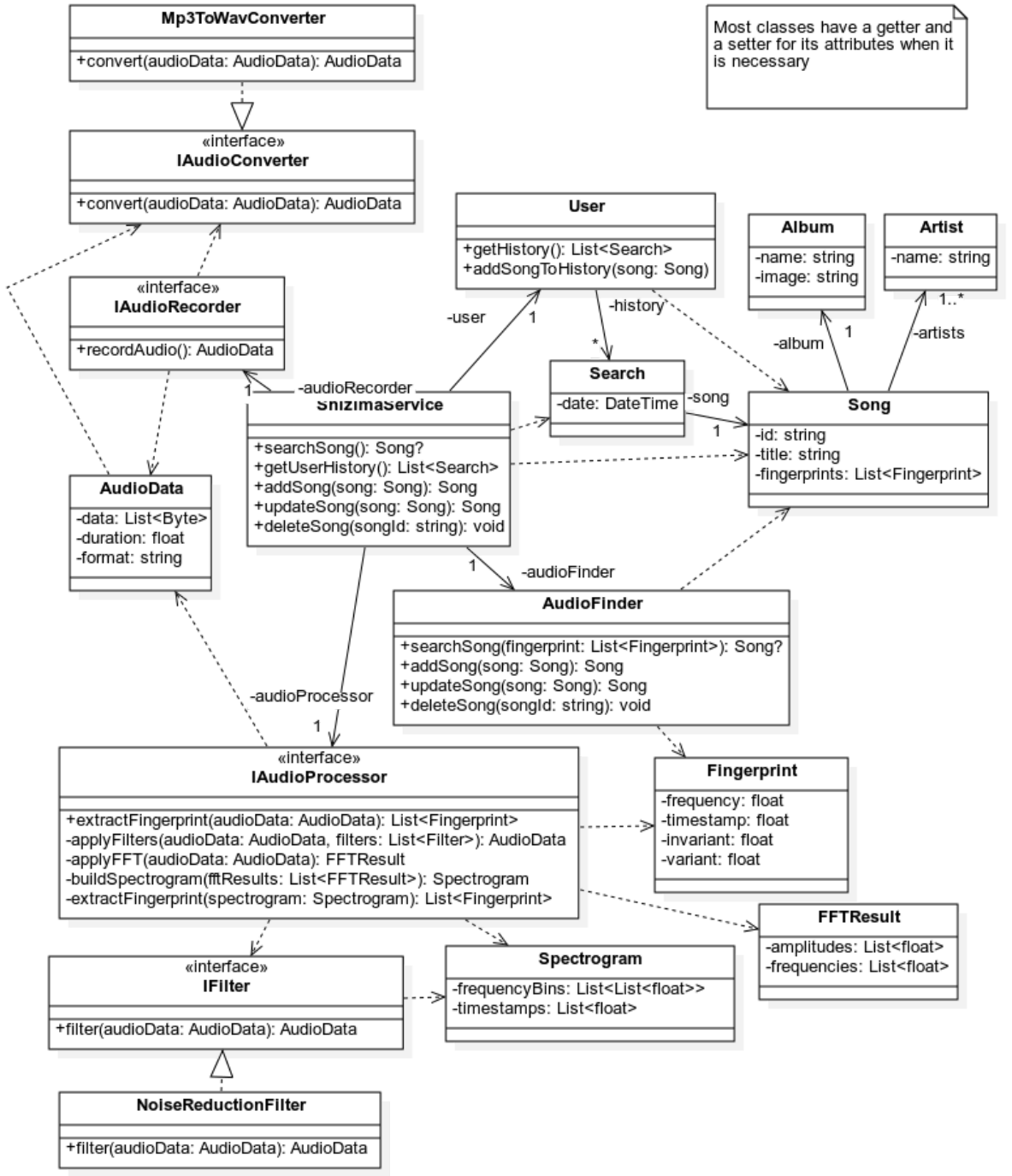
L'utilisateur peut :

- Identifier une chanson. Une fois cela fait, il peut consulter la chanson trouvée et son artiste.
- Consulter son historique. Depuis cet historique, il peut consulter une chanson trouvée récemment et son artiste.

Un administrateur peut :

- Faire les mêmes actions qu'un utilisateur normal.
- Gérer la base de données. C'est-à-dire, ajouter, modifier et supprimer une chanson

Diagramme de classes et Architecture



Fonctionnement principal

Nous avons découpé notre diagramme de classes afin qu'il tourne autour de la classe "ShizimaService" qui est issue du patron de conception "Façade". Cette façade rend accessible les fonctionnalités principales de l'application en masquant toute la complexité derrière. Ce service commencera par appeler l'AudioRecorder qui se chargera de capturer l'audio en temps réel et de le transformer en un objet AudioData. En fonction du type de

fichier audio retourné par l'AudioRecorder, notre AudioConverter pourra être appelé qui se chargera de convertir l'audio en .wav. Le fait d'utiliser ici des interfaces garantit que, par exemple, la conversion audio (par exemple, de MP3 à WAV) peut être facilement remplacée ou étendue sans modifier le reste du code. Ensuite, l'objet retourné contiendra les données brutes nécessaires aux traitements suivants.

Pour commencer, la création des empreintes se passe dans l'AudioProcessor avec plusieurs fonctions : une première fonction qui, en prenant un AudioData, se chargera d'appliquer les transformées de Fourier afin de nous rendre la liste des fréquences via un objet FFTResult. Ces mêmes fréquences seront ensuite transformées en spectrogramme pour en extraire les pics pour fabriquer nos empreintes. Si le besoin se pose, il sera possible d'appliquer des filtres au spectrogramme afin de nettoyer le son et garantir une extraction précise des empreintes. Ici aussi, il s'agit d'une interface dans le but d'étendre les possibilités sans modifier de code existant.

Pour finir, les empreintes audio générées seront transmises à l'AudioFinder qui se chargera de communiquer avec l'API qui se connectera à la base de données afin de rechercher des correspondances. Si une correspondance est trouvée, l'objet Song correspondant est renvoyé.

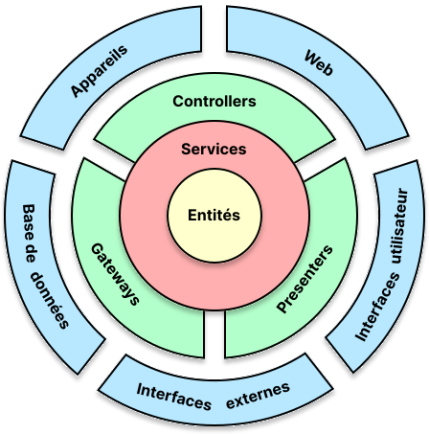
L'utilisateur sera aussi stocké en base de données grâce à l'api et l'authentification se fera sans connexion, à partir d'un token.

Principes SOLID

Nous avons essayé de respecter le plus possible les principes SOLID qui permettent d'avoir un code maintenable au fil du temps :

- Les fonctionnalités sont séparées dans des classes afin de respecter le principe "S" des principes SOLID: le fait qu'une classe ne possède qu'une seule responsabilité.
- En utilisant des interfaces, nous avons essayé de respecter au plus le principe "O" qui stipule qu'une classe doit être ouverte à l'extension mais fermée à la modification.

Clean architecture et API



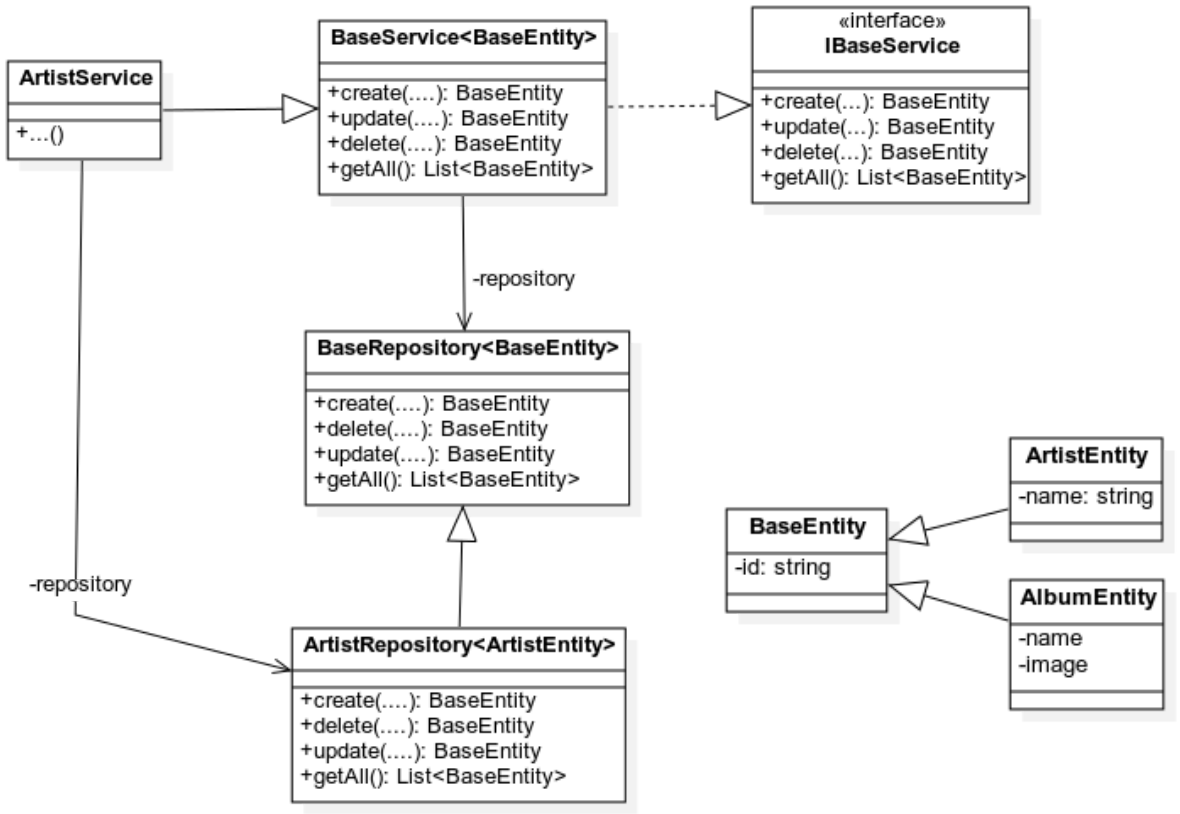
Pour le choix d'architecture de notre API, nous préférons nous orienter vers une architecture proche de la "Clean architecture" représentée par la figure ci-contre. Ce modèle permet de bien séparer les différentes parties du code, rendant ainsi la maintenance et les mises à jour plus faciles. Les couches (domaines, application, infrastructure) sont définies et bien séparées, ce qui améliore la testabilité et réduit les interactions complexes entre les composants. En isolant la logique métier des détails techniques, comme les bases de données ou les frameworks externes, cette architecture rend l'application plus flexible et prête à s'adapter aux futures modifications.

Pour être en accord avec cette architecture, la gestion des données est séparée en DTOs (Représentent les données à transmettre à l'interface utilisateur) et en entités (Modélisent les objets métier tels qu'ils sont persistés dans la base de données). Dans une application de plus grande envergure, nous pourrions avoir une couche supplémentaire : le "Domaine" qui permettrait de séparer l'entité de la logique métier. Les objets du domaine comprendront la logique métier et les différents algorithmes.

De même pour l'accès à la base de données qui se fera à partir d'un service. Il appellera un repository qui se chargera de communiquer avec la base de données. Pour chaque type de données à stocker, un nouveau service et un nouveau repository seront créés. Étant donné que nous stockons plusieurs autres types de données telles que les artistes, les albums et l'historique de l'utilisateur, il faudra aussi implémenter leurs services. Ces repository et services seront accompagnés d'un BaseRepository et d'un BaseService qui seront des classes génériques permettant de définir les opérations de base afin d'éviter de recopier du code plusieurs fois.

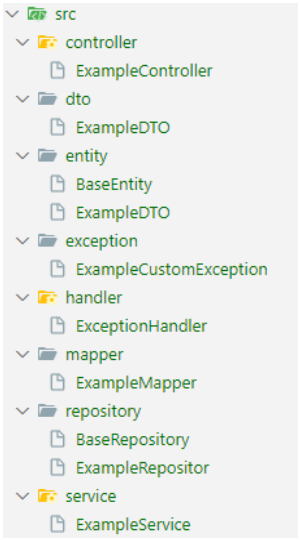
Ces services et repository seront suivis aussi d'un BaseEntity permettant de maximiser la généricité. Cela permet aussi de n'implémenter dans SongService que les méthodes spécifiques aux musiques.

Exemple des classes génériques dans cette architecture:

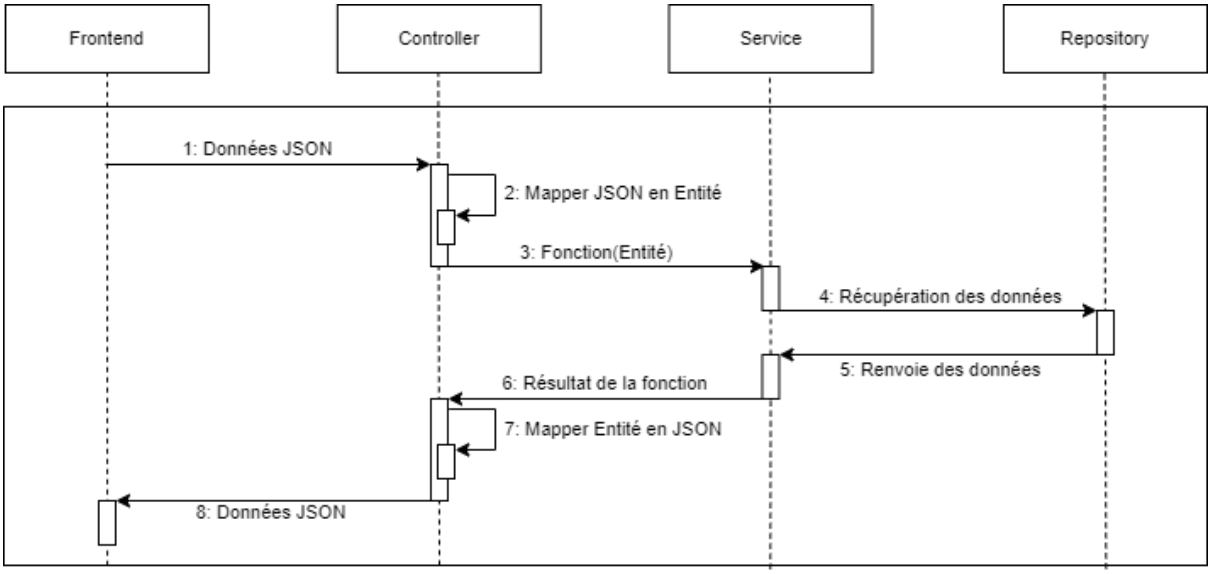


Ici seul l'artiste est représenté en guise d'exemple.

Exemple de structure que pourrait adopter l'api pour être conforme à la clean architecture où chaque objet possède son contrôleur, son service et son repository tout en utilisant les services et repository génériques pour éviter de dupliquer du code :



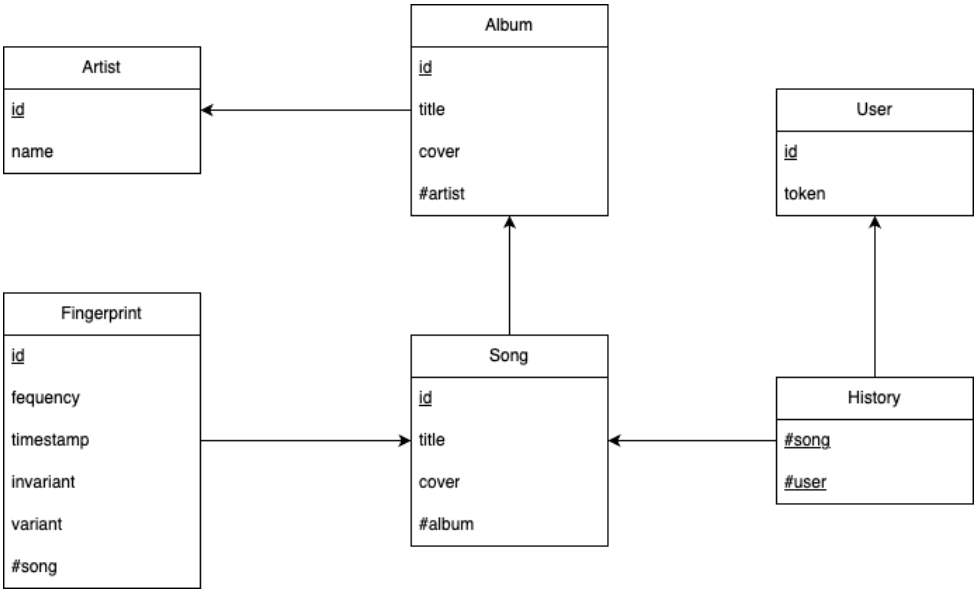
Exemple d'un flux de données dans l'api avec notre structure :



Exceptions personnalisées

Il sera aussi possible de mettre en place des exceptions personnalisées qui permettront d'informer précisément l'utilisateur ou de loguer les problèmes pour un diagnostic plus rapide.

Diagramme de base de données (MLD)



Dans la base de données, on va stocker toutes les informations liées aux musiques, aux albums, aux artistes et aux utilisateurs.

- Chaque chanson sera aussi liée à des empreintes afin de pouvoir rechercher des correspondances avec l'audio qui sera enregistré.

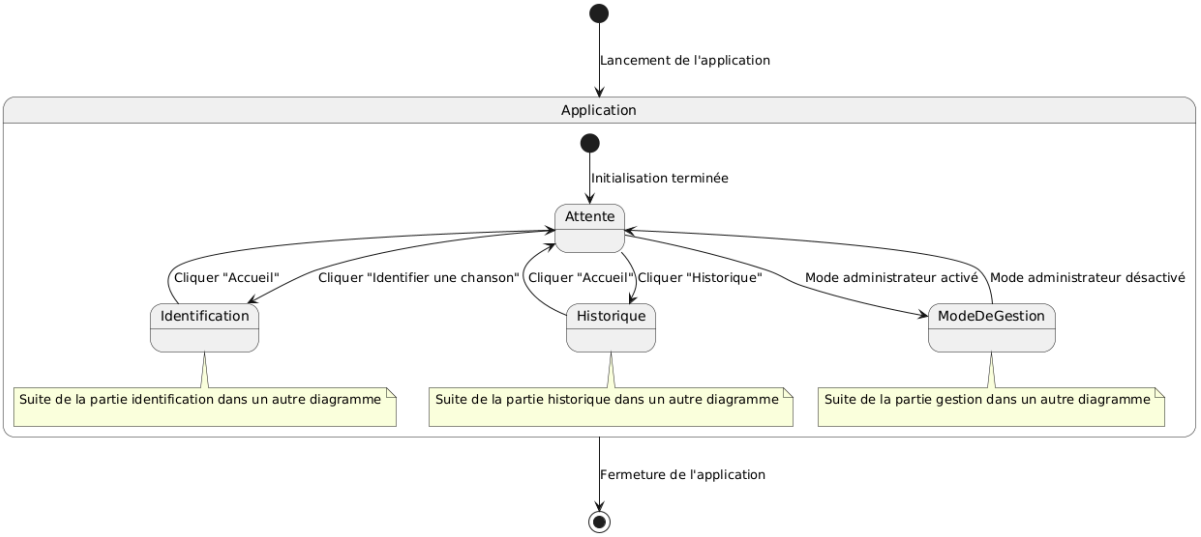
- Concernant l'utilisateur, nous ne stockons pas d'informations personnelles. On identifiera l'utilisateur grâce à son numéro d'appareil qui est unique pour chaque installation d'application. C'est-à-dire que si l'utilisateur supprime et réinstalle l'application, l'id change. Il aura aussi un token pour faire les appels à l'api. Ce token sera stocké en local dans le cache de l'utilisateur. Nous avons choisi de stocker l'historique dans notre base de données externe afin que si nous modifiions ou supprimons une information d'une musique, elle soit aussi modifiée chez l'utilisateur.

- Il y a enfin une table History qui permet de stocker l'historique de l'utilisateur. Il a besoin d'une table séparée car il y a une relation n,n entre User et Song.

Nous avons fait le choix de ne pas implémenter de compte utilisateur par connexion car il s'agit souvent d'une étape qui augmente le taux "Churn". Cependant, nous stockons les utilisateurs et leur historique en base de données, ce qui nous permettrait d'implémenter un système de connexion dans le futur si besoin.

Diagramme d’état

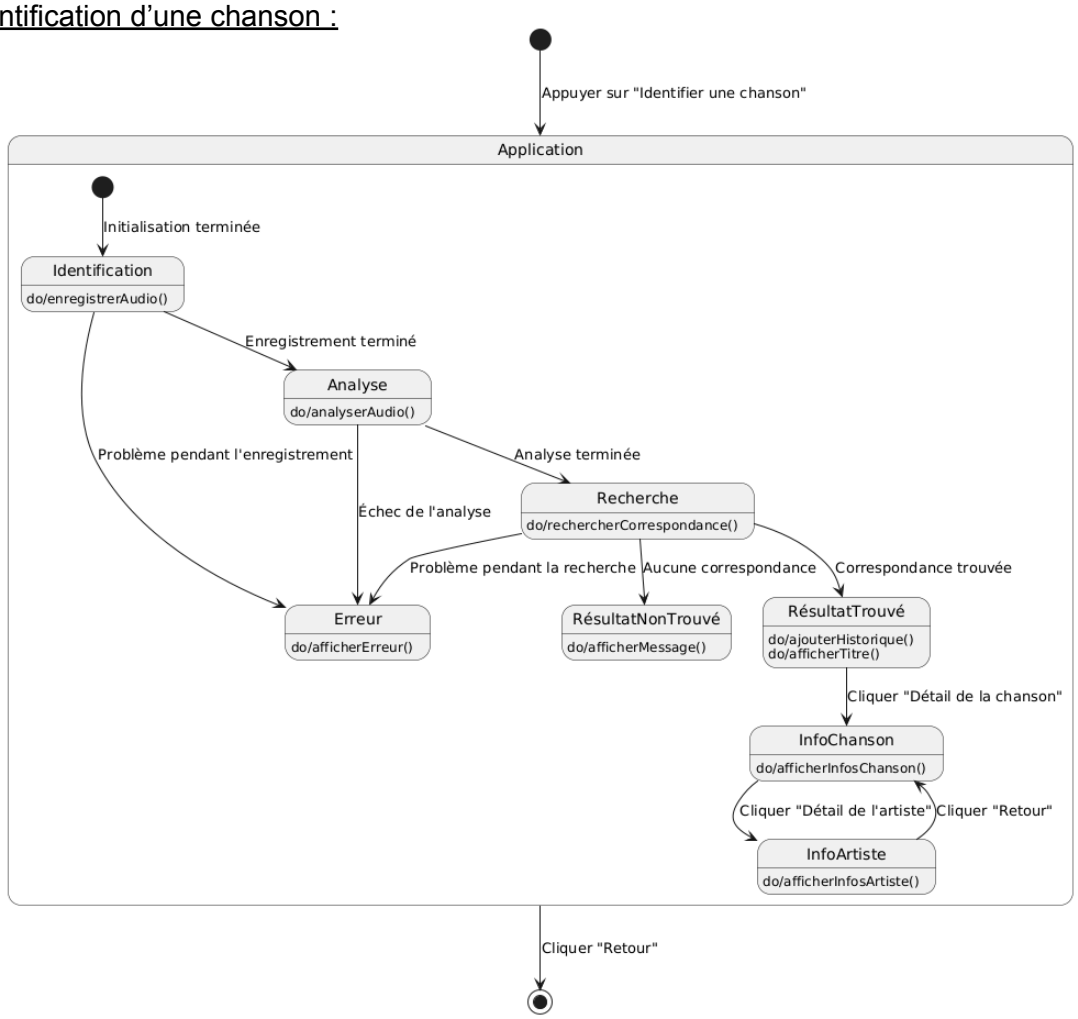
Diagramme général :



Le diagramme d’état se décompose en quatre grandes parties :

- Un état d’attente qui correspond à l’accueil de l’application dans lequel il ne se passe rien.
- L'identification correspond à l’état qui permettra à l’utilisateur de trouver une musique grâce à l’enregistrement d’un audio.
- L’historique, afin que l’utilisateur puisse retrouver les dernières chansons qu’il a identifiées.
- Le mode gestion, un mode réservé aux administrateurs afin d’ajouter, de modifier ou de supprimer des chansons dans la base de données.

L’identification d’une chanson :

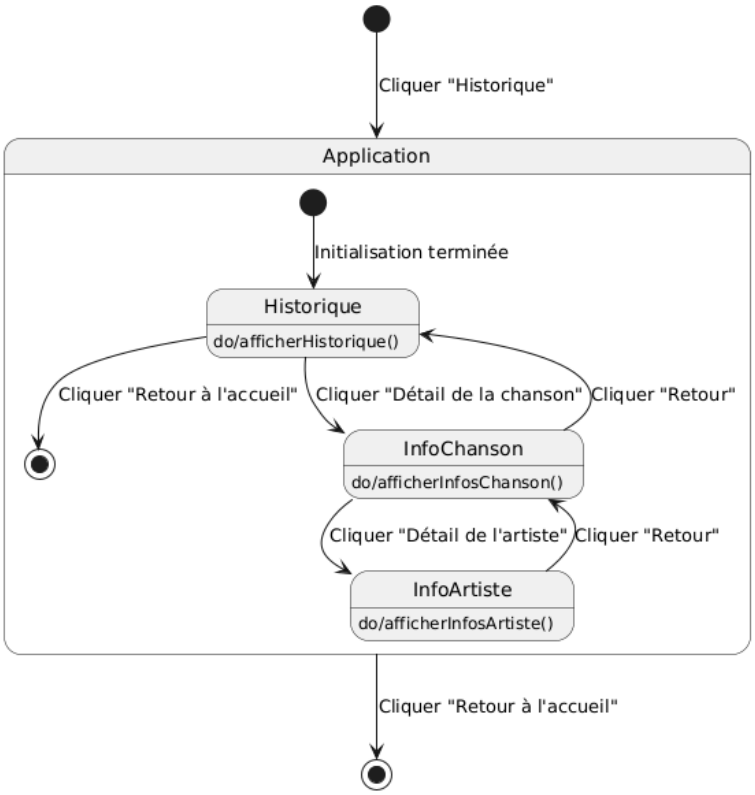


Lorsque l’utilisateur entre dans l’état d’identification, de nombreux états vont être déroulés sans que l’utilisateur s’en aperçoive. Tout d’abord, un enregistrement de l’audio va être fait. Puis, il sera analysé et comparé à la base de données afin de trouver une correspondance. À ce moment-là, l’utilisateur va être informé du résultat de cette recherche :

- Une musique trouvée.
- Pas de musique trouvée.
- Une erreur est survenue.

Dans le cas d’une musique trouvée, l’utilisateur a la possibilité de consulter les informations de la chanson ainsi que de son artiste.

Historique :



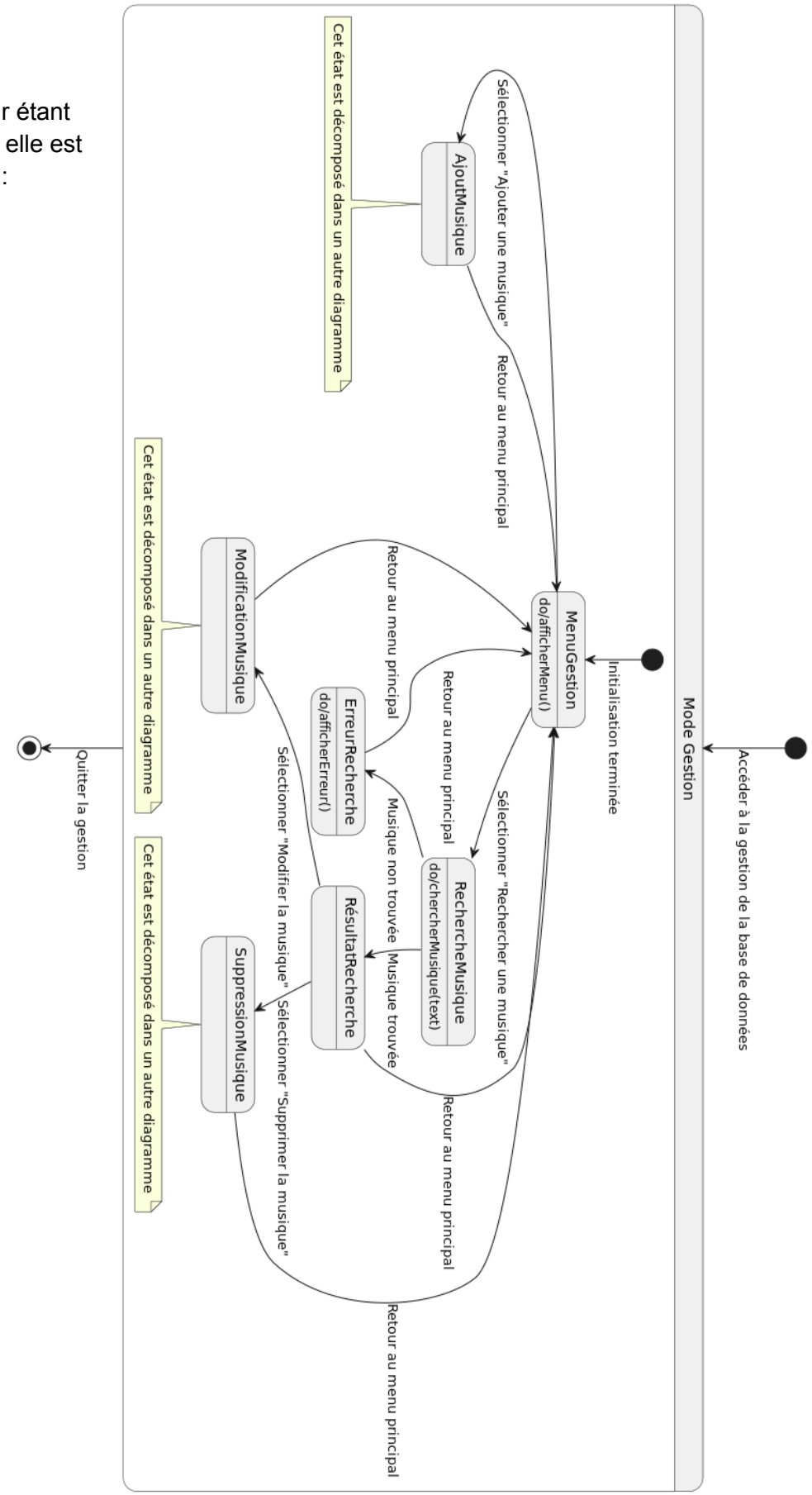
Nous avons choisi d'intégrer un historique de recherche afin que l'utilisateur puisse rapidement et facilement retrouver le titre d'une musique qu'il aurait cherché quelque temps auparavant.

Ainsi, l'utilisateur peut accéder à son historique de recherche depuis l'accueil et consulter les chansons ainsi que leur détail qu'il a identifiées.

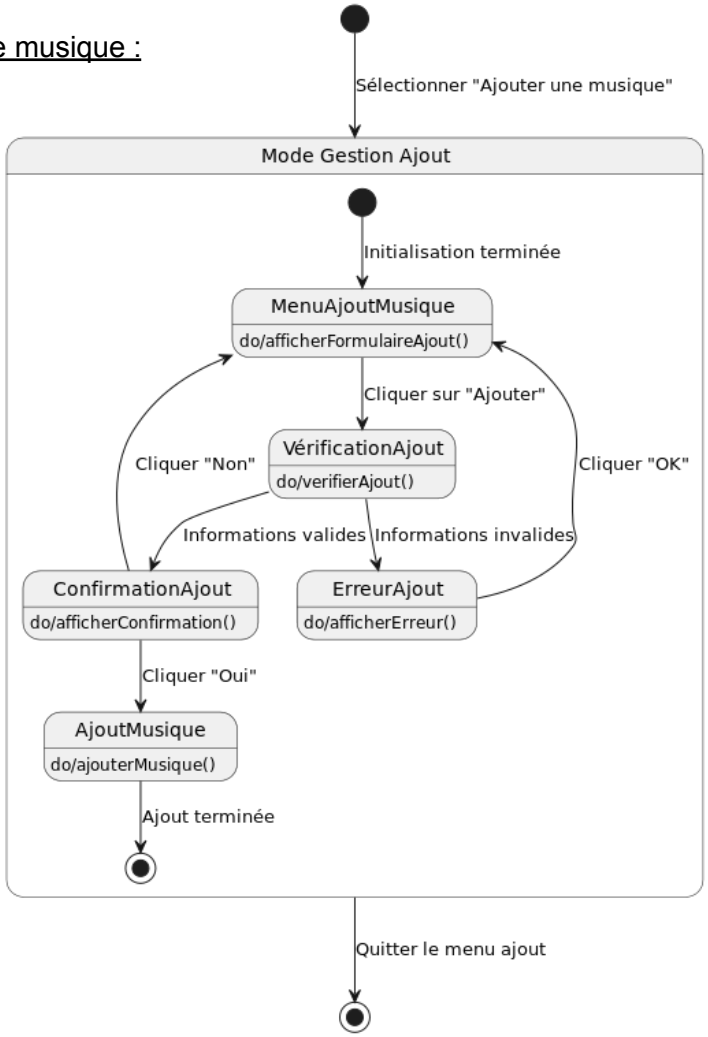
Section Admin :

La section administrateur étant elle aussi très complète, elle est détaillée en trois parties :

- L'ajout
- La modification
- La suppression

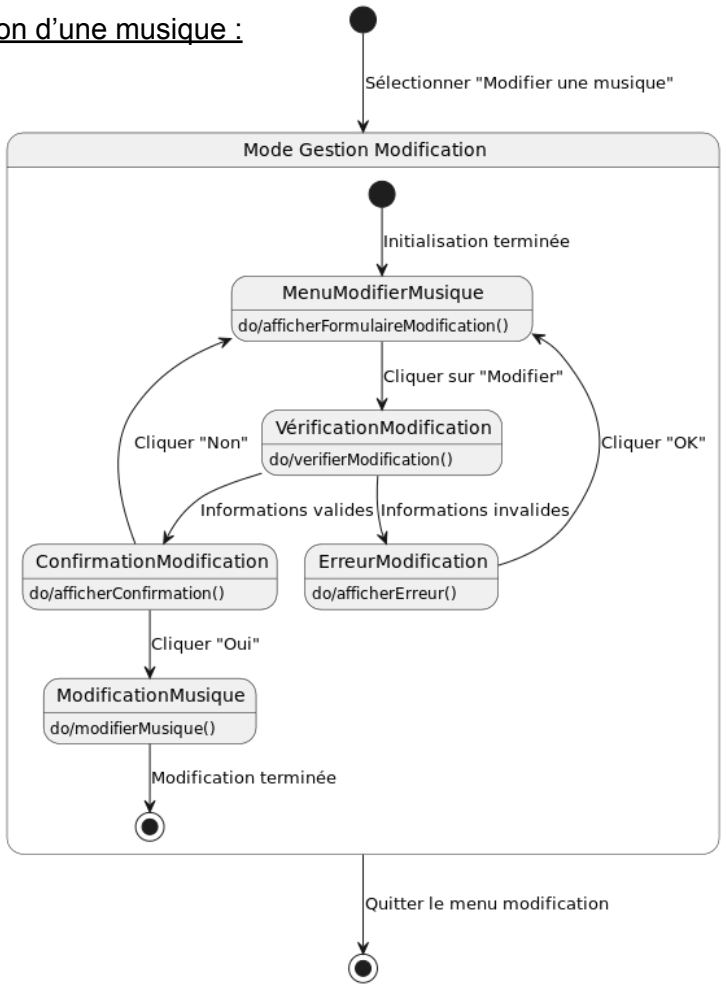


Section ajout d'une musique :



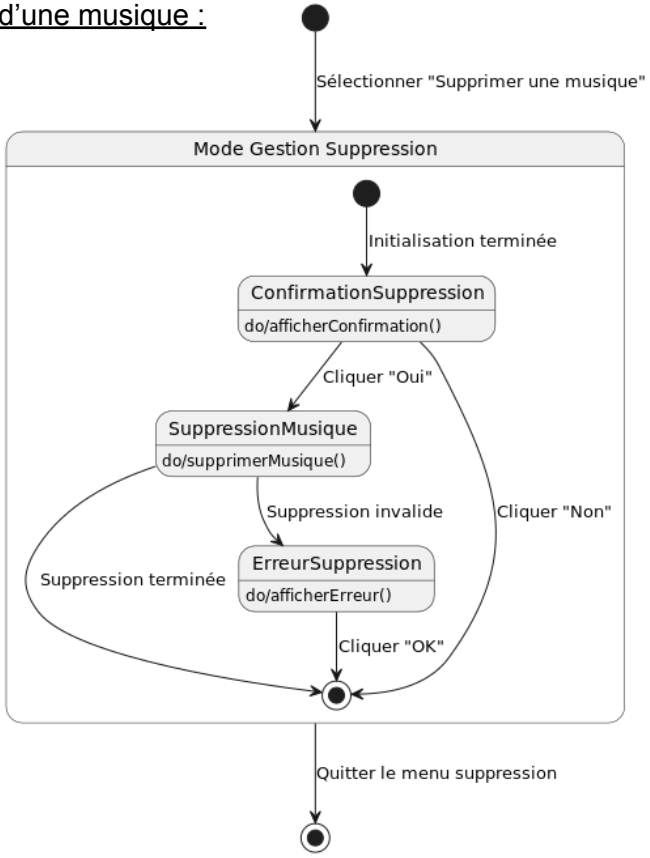
Pour ajouter une musique, l'administrateur devra passer par un formulaire. Ce formulaire sera vérifié dans le but de confirmer les informations entrées. Une gestion d'erreur sera aussi intégrée afin de traiter un éventuel problème.

Section modification d'une musique :



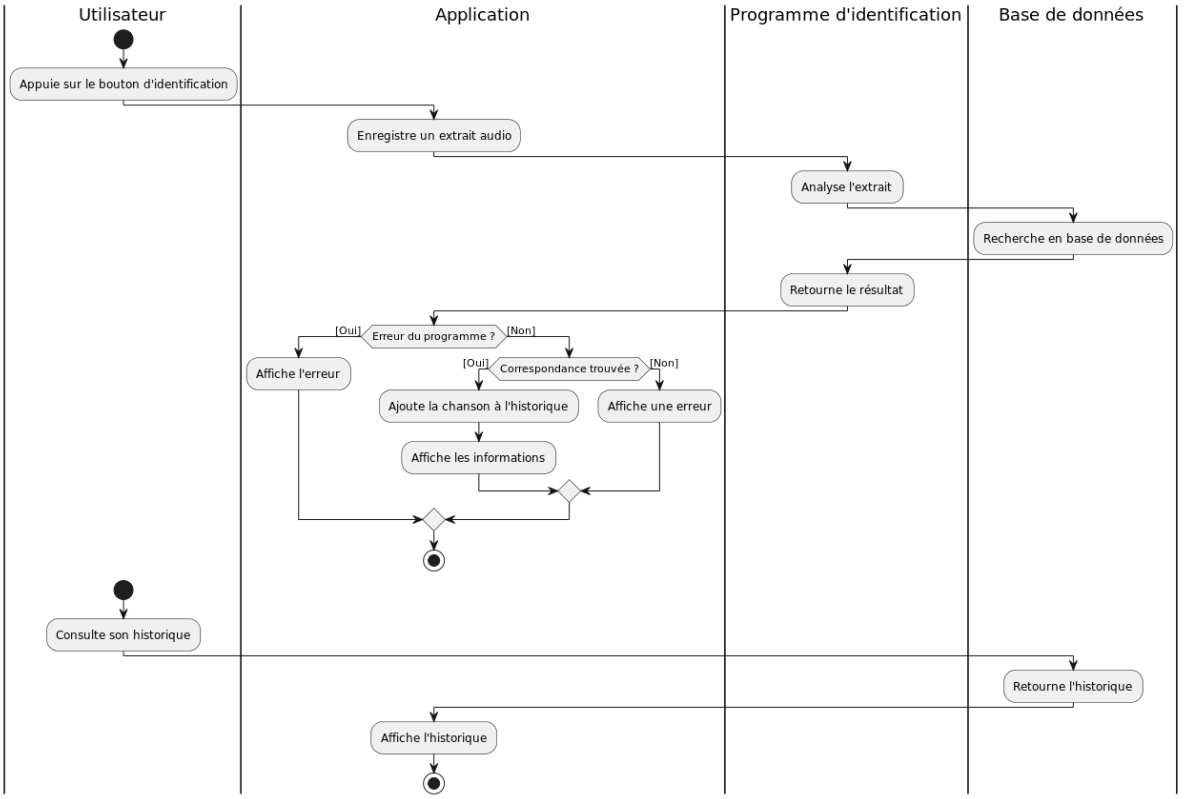
La modification d'une musique se rapproche beaucoup d'un ajout. Cela étant dit, le formulaire devra être pré rempli avec les informations de la musique sélectionnée.

Section suppression d’une musique :



La suppression d’une musique devra obligatoirement passer par une confirmation afin de ne pas faire d’erreur en supprimant la mauvaise musique.

Diagramme d’activité



Le diagramme d’activité permet de représenter les échanges entre les différents systèmes lors de l’identification d’une chanson et la consultation de l’historique.
Dans ce diagramme, nous présentons 2 cas: l’identification d’une musique ainsi que la consultation de l’historique de l’utilisateur.

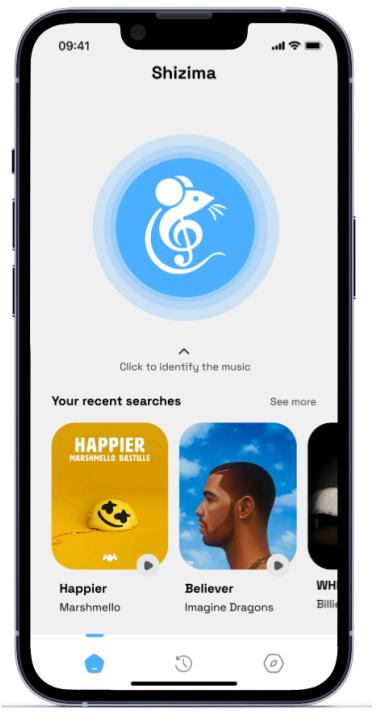
Direction artistique

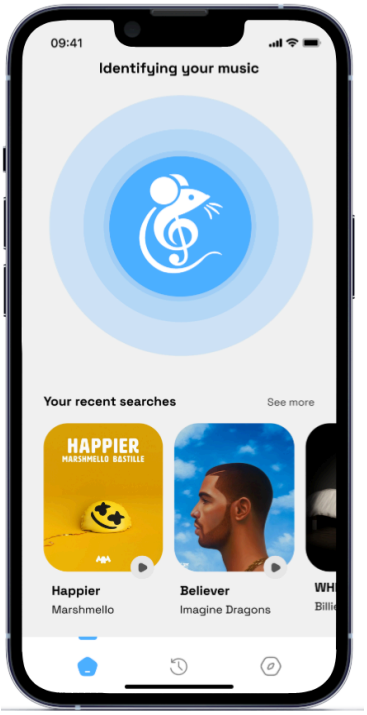


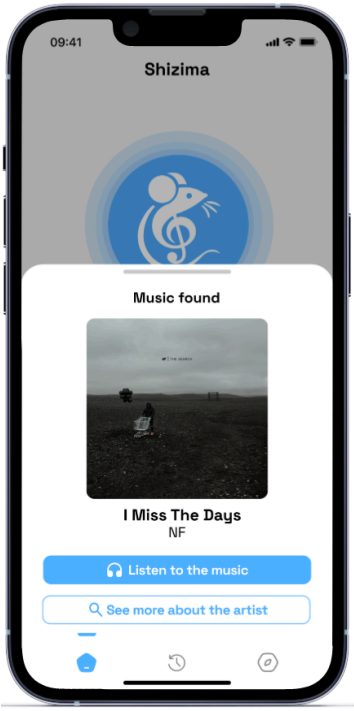
Logo de l'application présent sur l'icône de l'application ainsi que sur le bouton d'identification de musique.
Ce logo représente une souris, symbole de l'Isima, mélangée à une clé de sol

Police utilisée : **Space Grotesk**

Couleur HEX	Rendu	Description
#4DAFFF		Couleur d'accentuation principale
#393939		Couleur des sous-titres et des icônes
#E6E6E6		Couleur d'arrière-plan des boutons
#FAFAFA		Couleur d'arrière-plan des pages, proche du blanc. La barre de navigation quant à elle est entièrement blanche (#FFF) afin de créer une séparation entre les deux composants.

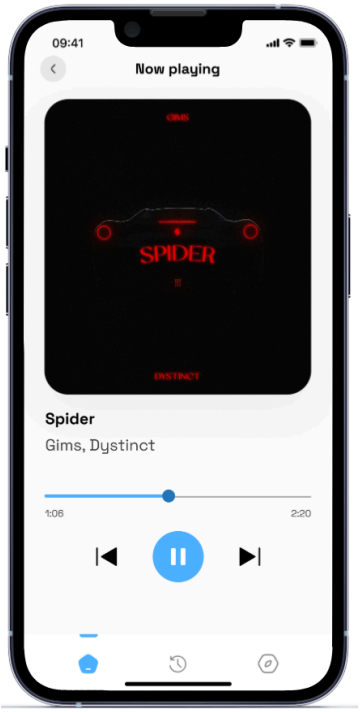




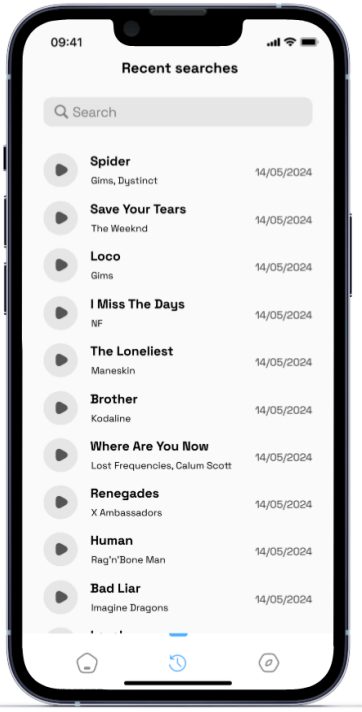


Page d'accueil **Identification d'une musique** **Résultat trouvé**

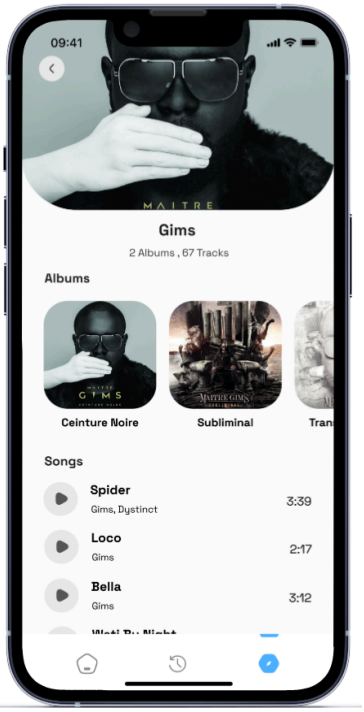
Le bouton d'identification possède un effet de pulsation qui s'agrandit lorsqu'on identifie.



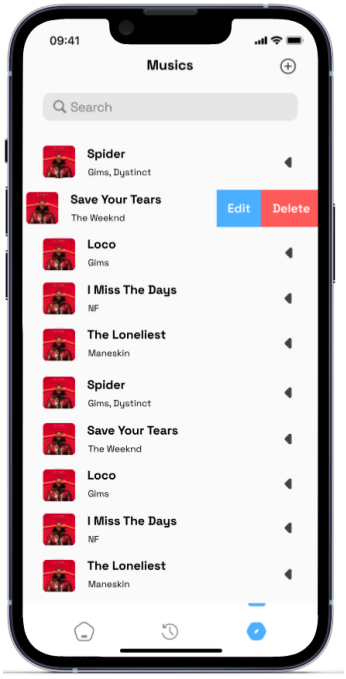
Lecture d'une musique



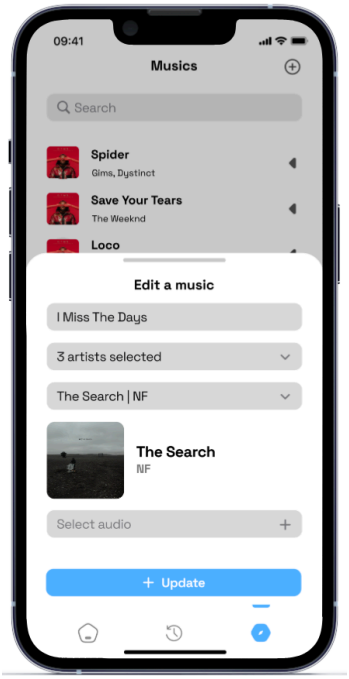
Historique



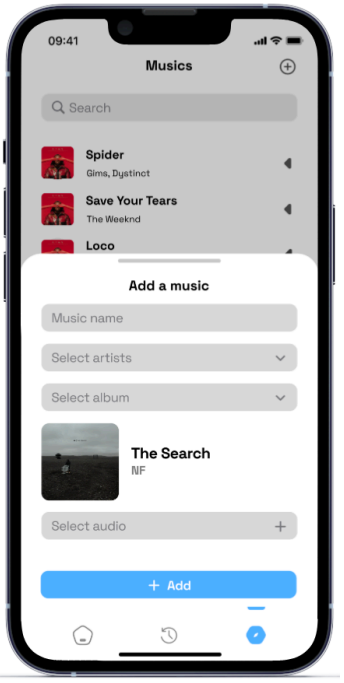
Page Artiste



Page Admin



Modification



Ajout